

Pseudonimización criptográfica de identificadores en

CMAT-research

Versión técnica

Centro de Aprendizaje de Matemáticas (CMAT)

1. Qué hace realmente el repositorio

CMAT-research no cifra los identificadores mediante un esquema reversible; los **pseudonimiza de manera determinística con HMAC-SHA256**. La implementación principal está en `cmat_analysis/src/cmat_analysis/privacy.py` y se aplica a las bases mediante `create_anonymized_release.py`.

La distinción es importante. En un cifrado existe, para quien posee la clave adecuada, una operación de descifrado que recupera el texto original. En HMAC no existe esa operación: a partir del valor producido no se “descifra” el ID. La forma correcta de expresar la garantía es que, **sin la llave secreta**, recuperar o incluso comprobar candidatos al identificador original a partir del pseudónimo es computacionalmente impracticable bajo los supuestos estándar de seguridad de HMAC-SHA256.

2. Un mecanismo criptográfico ampliamente utilizado

HMAC no es una construcción ad hoc del CMAT. Fue estandarizado en RFC 2104 y HMAC-SHA256 forma parte de numerosos sistemas de seguridad reales. Por ejemplo:

- **AWS Signature Version 4** utiliza AWS4-HMAC-SHA256 para autenticar solicitudes a servicios de Amazon Web Services.
- **GitHub** recomienda HMAC-SHA256 para verificar que los webhooks provienen de GitHub y que su contenido no fue alterado.
- El estándar **JSON Web Algorithms** define HS256 como HMAC usando SHA-256, utilizado en firmas/MAC de JSON Web Tokens y JSON Web Signatures cuando se emplea una clave compartida.
- **IPsec, IKE e IKEv2** tienen variantes estandarizadas basadas en HMAC-SHA256 para autenticación, integridad y funciones pseudoaleatorias.
- Diversas suites de **TLS 1.2** históricamente estandarizaron HMAC-SHA256 como código de autenticación de mensajes.

Esto no significa que AWS, GitHub, JWT, IPsec y CMAT protejan exactamente el mismo tipo de dato o sigan el mismo protocolo completo; significa que **utilizan la misma primitiva criptográfica madura y ampliamente analizada**, HMAC con SHA-256, para construir valores dependientes de un secreto que un tercero sin la clave no puede reproducir eficientemente.

3. Ejemplo completo con un ID

Supóngase el ID

$$x = 175199.$$

Para evitar cualquier confusión, la siguiente llave es **ficticia y se usa solamente para esta demostración**:

$$K = 7b41e7a1c76f6b50f58a0c628a3f0e8daed7ad4f6dd6c7e97ad31e8f8ad1e933$$

La llave real del proyecto no debe aparecer en documentación, datos compartidos, Git, artefactos de ejecución ni archivos de resultados.

El repositorio incorpora además un espacio de nombres. Para estudiantes se utiliza `stu`, por lo que el mensaje que realmente entra a HMAC es

$$m = \text{stu}:175199.$$

Con la llave de demostración anterior, HMAC-SHA256 produce el resumen completo

$$\text{c504a778e1e4b331721d7c88f22c65e0008707cede372864628d04a20bbe27fb}$$

El código conserva los primeros 32 caracteres hexadecimales y antepone el espacio de nombres:

$$\boxed{\text{stu_c504a778e1e4b331721d7c88f22c65e0}}.$$

Por tanto, en una base de investigación ya no aparece 175199. Aparece una etiqueta estable que permite reconocer al mismo estudiante en distintas tablas sin guardar su identificador institucional.

4. Por qué la llave es tan larga

El script recomienda generar la llave mediante:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

`token_hex(32)` genera 32 bytes aleatorios y los representa mediante 64 caracteres hexadecimales. Cada dígito hexadecimal codifica 4 bits, de modo que

$$64 \times 4 = 256 \text{ bits.}$$

Una llave aleatoria de 256 bits tiene

$$2^{256} \approx 1.16 \times 10^{77}$$

valores posibles. Incluso suponiendo de forma irrealista que un atacante pudiera probar

$$10^{18}$$

llaves por segundo de manera sostenida, recorrer todo el espacio requeriría aproximadamente

$$3.7 \times 10^{51} \text{ años.}$$

La longitud por sí sola no basta: la clave debe ser **aleatoria y secreta**. Una cadena larga pero predecible, por ejemplo una frase conocida, no ofrece la misma seguridad.

5. Cómo funciona HMAC por dentro

Sea $H = \text{SHA256}$, sea m el mensaje y sea K' la llave ajustada al tamaño de bloque de SHA-256. HMAC calcula

$$\text{HMAC}_H(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m)),$$

donde $\parallel$ representa concatenación, $\oplus$ es XOR, e `ipad` y `opad` son dos patrones constantes distintos definidos por el estándar.

Conceptualmente hay dos capas. Primero se mezcla la llave con un patrón interno y se calcula un hash del mensaje; después ese resultado se vuelve a combinar con la llave, ahora mediante un patrón externo, y se calcula SHA-256 otra vez. Esta estructura evita las debilidades que tendría una construcción ingenua del tipo $\text{SHA256}(K \| m)$ y cuenta con un análisis criptográfico formal desarrollado específicamente para HMAC.

En CMAT-research la función relevante es esencialmente:

```
message = f"{namespace}:{canonical_identifier(value)}".encode("utf-8")
digest = hmac.new(key, message, hashlib.sha256).hexdigest()
return f"{namespace}_{digest[:32]}"
```

La canonicalización evita que un identificador numérico leído como entero y el mismo identificador leído como flotante integral produzcan representaciones diferentes. El espacio de nombres, por su parte, realiza *domain separation*: el mismo valor puede producir pseudónimos independientes como `stu_...`, `prof_...` o `advisor_...`

6. Determinismo, truncamiento y colisiones

HMAC es determinístico: con el mismo ID, la misma llave y el mismo espacio de nombres se obtiene exactamente el mismo valor. Esa propiedad permite enlazar registros longitudinales.

El HMAC completo de SHA-256 tiene 256 bits, pero el repositorio conserva 32 caracteres hexadecimales, es decir, 128 bits. Para N pseudónimos distintos, la probabilidad aproximada de al menos una colisión accidental es, mediante la aproximación de cumpleaños,

$$\Pr(\text{colisión}) \approx \frac{N(N-1)}{2^{129}}.$$

Incluso para $N = 10^6$ registros distintos, esta probabilidad es del orden de 10^{-27} , despreciable para la escala del proyecto.

7. ¿Es “indescifrable”?

En lenguaje informal puede decirse que el pseudónimo **no se puede descifrar**, porque HMAC no es cifrado y no existe una función inversa que recupere el ID. Técnicamente, la afirmación más precisa es:

Sin la llave secreta, obtener o verificar eficientemente el identificador original a partir del pseudónimo es computacionalmente impracticable; con la llave, en cambio, pueden probarse candidatos conocidos.

Esto último es esencial. Si alguien obtuviera simultáneamente la llave secreta y una lista plausible de IDs institucionales, podría calcular HMAC para cada ID y comparar los resultados. Por ello la seguridad práctica depende de mantener la llave separada de los datos.

8. Qué protege y qué no protege

La transformación protege **identificadores directos**, pero no convierte automáticamente el conjunto de datos en anónimo. Fechas, carrera, semestre, materia, tema, calificaciones y patrones longitudinales pueden actuar como cuasi-identificadores y permitir inferencias mediante información externa.

Por ello, el esquema completo de CMAT combina

HMAC-SHA256 + llave secreta externa + control de acceso + minimización de datos

y debe describirse como **pseudonimización**, no como anonimización absoluta.

9. Referencias técnicas

- RFC 2104, *HMAC: Keyed-Hashing for Message Authentication*: <https://www.rfc-editor.org/rfc/rfc2104>
- AWS, *Signature Version 4*: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_signing-elements.html
- GitHub, *Validating webhook deliveries*: <https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries>
- RFC 7518, *JSON Web Algorithms (JWA)*: <https://www.rfc-editor.org/rfc/rfc7518>
- RFC 4868, *Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec*: <https://www.rfc-editor.org/rfc/rfc4868>